

SIMATS ENGINEERING ASSESSMENT TOOL 2

COURSE CODE: CSA02

COURSE NAME: C Programming

COURSE OUTCOME COVERED

CO5: Design modular and scalable C programs using functions, recursion, pointers, and file handling. (BL4 and BL5)

Assessment Tool Weightage: 20%

QUESTIONS:

Total Marks: 100

Annexure A



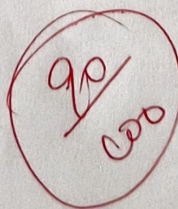
Debugging & Optimisation Task

Code of Conduct:

I K.kavya-192425283 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



Debugging the following code

```
#include <stdio.h>
int main()
{
    int i = 0;
    do {
        printf("%d", &i); i++;
    }
    while (i < 5);
    return 0;
}
```

Q17 Debugging the following code

```
#include <stdio.h>
int main() {
    int i;
    for (i = 1; i < 10; i++) {
        printf("%d", i);
    }
    printf("%d\n", i);
    return 0;
}
```

Q5: Debugging

```
#include <stdio.h>
int main()
{
    int age = 15;
    if (age > 18 || age < 12) {
        printf("Eligible");
    }
    else {
        printf("Not eligible");
    }
}
```

```
y  
return 0;  
y
```

Output : Not eligible

Debugging

```
Q4 #include <stdio.h>
```

```
int main()
```

```
{
```

```
char ch = 'A';
```

```
printf("character is %c\n", ch);
```

```
return 0;
```

```
}
```

Output: character is A

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Identification	20%	Accurately identifies all errors and root causes with clear understanding	Identifies most errors with minor gaps	Identifies some errors but misses key issues	Unable to identify errors correctly
Debugging	25%	Applies systematic debugging techniques effectively to resolve issues	Uses appropriate debugging methods with minor errors	Limited debugging approach; requires guidance	Unable to debug or resolve issues
Optimization	20%	Optimizes code by significantly improving time complexity using efficient algorithms	Improves time complexity with minor gaps in efficiency	Limited improvement in time complexity	No improvement; inefficient approach retained
Analysis	20%	Demonstrates strong logical reasoning and clear justification of solutions	Shows reasonable analysis with some justification	Limited analysis; weak reasoning	No analytical reasoning demonstrated
Code Quality	15%	Produces clean, well-structured code with proper comments and documentation	Code is mostly clear with minor documentation gaps	Code lacks structure and proper documentation	Poorly written code with no documentation